Market-Basket Analysis on the IMDB Top-1000 Dataset

Frequent itemsets with A-Priori, PCY and SON

Abdelrahman Ibrahim , Mat.Number : 52178A

Abstract

We look for groups of actors that often appear together in the same movie, using the IMDB Top-1000 dataset: each movie is a basket and each of its (up to) four listed stars is an item. We implement A-Priori, PCY and SON in plain Python, check them against a brute-force count, derive association rules (confidence and interest), and implement SON on Spark. For $s = 3$ all methods find the same 299 frequent itemsets; PCY counts 104 pairs instead of 622 for A-Priori, and SON examines 13 492 candidates. Scalability is tested on a second, larger real dataset (24 402 movies). Code:https://github.com/Abdelrahman-SS/Market-basket-analysis.

1 Dataset

Main dataset. *IMDB Dataset of Top 1000 Movies and TV Shows* from Kaggle (file `imdb_top_1000.csv`), downloaded with the Kaggle API (the credentials in the submitted notebook are replaced by `xxxxxx`). It has 1000 rows, one per movie. Only the columns `Star1`–`Star4` are used, because the task defines the baskets through the stars; all other columns are ignored. A movie lists at most four stars, so the largest possible itemset has four items.

Dataset for the scalability test. *25k Movie Dataset* from Kaggle (CC BY-NC-SA 4.0), which has 24 402 movies with a non-empty column `Top 5 Casts`. The column stores a list of up to five names as text, for example `"['A', 'B']"`.

2 Data organisation

A movie is a *basket* and a name is an *item*. Each basket is a sorted Python list of names and the dataset is a list of baskets. Sorting guarantees that the same pair always has the same order, so (A, B) and (B, A) are never counted separately. Itemsets are stored as tuples of names, which can be dictionary keys, and every algorithm keeps its counters in dictionaries: itemset $\rightarrow$ number of baskets. Only the itemsets that actually occur are stored, not all possible pairs. Item names are kept as strings and are not mapped to integers a, which is simpler but uses more memory.

3 Pre-processing

- Names are stripped of surrounding spaces and missing values are skipped.
- A basket is a set: a name repeated in the same movie counts once.
- Names are used as they are, without fuzzy matching.
- For the 25k dataset the text of `Top 5 Casts` is converted to a list with `ast.literal_eval`, and rows with a missing cast are dropped.
- A global switch `USE_SAMPLE` selects a random sample of `SAMPLE_SIZE` baskets instead of the full data; the seed is fixed (42).

4 Algorithms

Let s be the support threshold: an itemset is *frequent* if it appears in at least s baskets. All algorithms use monotonicity: every subset of a frequent itemset is frequent.

A-Priori Level 1 counts every item and keeps the frequent ones. At level k we generate the k-subsets of each basket and count a subset only if all its $(k-1)$-subsets were frequent at the previous level. The algorithm stops when a level has no frequent itemset.

PCY In the first pass, besides the item counts, every pair of every basket is hashed to one of $B = 10\,000$ counters (CRC32 of the two names joined by "|", modulo B); the pair itself is not stored. Between the passes the counters become a bitmap: a bucket is frequent if its count is at least s. In the second pass a pair is counted only if both items are frequent and its bucket is frequent. A bucket counts at least as much as any single pair in it, so a bucket below s proves that none of its pairs is frequent: no frequent pair is lost. Collisions can only make us count pairs we did not need. Only pairs are hashed; triples and larger sets are found as in A-Priori.

SON The baskets are split into $m = 5$ chunks. In pass 1 each chunk is mined with A-Priori and the lower threshold $\lceil s \cdot |\text{chunk}|/n \rceil$ (n is the total number of baskets); the union of the results is the candidate set. In pass 2 the candidates are counted on all the baskets and those with count at least s are kept. A globally frequent itemset must reach its share in at least one chunk, so there are no false negatives, and pass 2 removes the false positives. The result is exact.

Association rules. For a frequent itemset J and an item $j \in J$, the rule $J \setminus \{j\} \to j$ has

$$\text{conf} = \frac{\text{supp}(J)}{\text{supp}(J \setminus \{j\})}, \qquad \text{interest} = \text{conf} - \frac{\text{supp}(\{j\})}{N},$$

where N is the number of baskets. By monotonicity the supports are already available, so no extra pass is needed.

Brute force. For correctness we count every subset of every basket without pruning and filter by s at the end. This is not an algorithm from the book (Sec. 6.2.6 only speaks of avoiding an "exhaustive count of all sets"); we use it as a reference answer, which is feasible because a basket has at most 15 (4 stars) or 31 (5 names) non-empty subsets.

Spark. Spark runs in local mode (`local[*]`). Counting items and pairs uses `flatMap` to emit $(\text{itemset}, 1)$, `reduceByKey` to add the counts and `filter` to keep counts $\geq s$. SON on Spark uses one partition per chunk: `mapPartitions` runs A-Priori with the local threshold on each partition and `distinct` merges the candidates (pass 1); the candidates are broadcast, every basket emits the candidates it contains, and `reduceByKey` counts them (pass 2).

5 Scalability

- **Memory.** Only counters are kept in memory. A-Priori counts a pair only if both items are frequent, and PCY also requires a frequent bucket; the bucket array has a fixed size B that does not depend on the data. The pair counts are what needs the most memory (Sec. 6.2.4), which is why PCY filters pairs.
- **Passes.** A-Priori and PCY make one pass per level over the baskets; SON makes exactly two passes.

- **Parallelism.** The chunks of SON are independent, so it maps to two MapReduce rounds (Sec. 6.4.4), which our Spark version implements. The same code could run on a cluster.
- **Limitation.** All baskets are held in a Python list, so the data itself must fit in memory. To go beyond that, the baskets would be read from a file on every pass.
- **Measurement.** The Top-1000 file has only 1000 baskets, so we measure the growth of the running time on the 25k dataset, on random samples of 1 000, 5 000, 10 000 and 24 402 baskets (Table 4).

6 Experiments and results

6.1 Correctness on the Top-1000 data

For $s = 3$, A-Priori, PCY, SON and the brute-force count return exactly the same dictionary (itemsets and counts): 299 frequent itemsets. They are 271 single actors, 25 pairs and 3 triples (Table 1). The Spark counts of items and pairs match the plain version (271 items, 25 pairs), and Spark SON returns the same 299 itemsets.

Table 1: Top-1000 data: number of frequent itemsets by size for different thresholds(A priori algorithm).

s	size 1	size 2	size 3	size 4	total
2	641	121	25	4	791
3	271	25	3	0	299
4	145	5	1	0	151
5	79	3	1	0	83
8	17	0	0	0	17

6.2 Work done by the algorithms ($s = 3$)

Table 2: Counters used by each method on the Top-1000 data, $s = 3$.

Distinct pairs counted by A-Priori (both items frequent)	622
Distinct pairs counted by PCY (both items and bucket frequent)	104
Frequent pairs (same result for both)	25
SON candidates (local threshold 1, 5 chunks of 200)	13 492
Frequent itemsets (all sizes)	299

6.3 Association rules

Rules built from the frequent itemsets for $s = 3$, sorted by confidence (Table 3). The strongest rules correspond to recurring casts of franchises: *The Lord of the Rings*, *Star Wars* and *Harry Potter*.

Table 3: Some of the top rules ($s = 3$).

Rule	support	confidence	interest
Ian McKellen, Orlando Bloom → Elijah Wood	3	1.00	0.997
Carrie Fisher, Harrison Ford → Mark Hamill	3	1.00	0.997
Mark Hamill → Carrie Fisher	3	1.00	0.996
Elijah Wood → Orlando Bloom	3	1.00	0.996
Rupert Grint → Daniel Radcliffe	6	1.00	0.994
Emma Watson, Rupert Grint → Daniel Radcliffe	5	1.00	0.994

6.4 Scalability on the 25k dataset

Table 4: Running time in seconds on random samples of the 25k dataset, with $s = \max(3, \lfloor 0.0005\, n \rfloor)$. All three methods returned the same result in every row. Each time is from a single run, rounded to 0.1 s.

baskets	s	itemsets	A-Priori	PCY	SON
1 000	3	84	0.0	0.0	0.1
5 000	3	1 532	0.1	0.1	0.3
10 000	5	1 496	0.1	0.2	0.7
24 402	12	1 232	0.3	0.4	0.5

6.5 Spark against plain Python

Table 5: SON in plain Python and on Spark (local mode) on the same samples. The two methods returned the same result in every row.

baskets	s	plain SON (s)	Spark SON (s)
1 000	3	0.11	2.03
5 000	3	0.50	6.74
10 000	5	0.94	2.96
24 402	12	0.50	1.72

7 Discussion

Correctness. The three algorithms and the brute-force count agree on the Top-1000 data for $s = 3$ (299 itemsets), and they agree with each other on every sample of the 25k dataset. This supports the correctness of our implementations.

Effect of the threshold. At $s = 2$ there are 641 actors that appear in at least two movies; at $s = 3$ there are 271. Most actors appear in only one or two movies, so the results are very sensitive to s. A frequent group of four also contributes its triples and pairs, so the counts of different sizes are not independent.

PCY. Both A-Priori and PCY count only pairs of frequent actors, but PCY adds the bucket filter and counts 104 pairs instead of 622 (518 fewer, about 83%), finding the same 25 frequent pairs. Of the 104 pairs, 79 are counted without being frequent: they pass the filter because their bucket is shared with other pairs or has a total of at least 3. Most pairs occur in only one or two movies, so most buckets stay below s and the filter works well even on this small dataset. The

price is an extra pass of hashing in pass 1: on the 25k dataset PCY is not faster than A-Priori in running time (0.4 s against 0.3 s for 24 402 baskets), so its benefit is memory and not speed. We used $B = 10\,000$ throughout and did not tune it.

SON. With 5 chunks of 200 movies and $s = 3$ the local threshold is 1, so every itemset that occurs in a chunk becomes a candidate: 13 492 candidates for 299 frequent itemsets, about 45 times more. The result is exact, but pass 2 does more counting than A-Priori, which is why SON is the slowest or close to it in Table 4. Its benefit is that the chunks are independent and can be processed in parallel.

Association rules. The rules with the highest confidence come from franchises with a recurring cast (Table 3). A confidence of 1 with a support of 3 rests on only three movies, so it should be read with caution; the interest is close to 1 mainly because these actors are rare.

Scalability. For a 24 times larger dataset (1 000 to 24 402 baskets) the running times of A-Priori, PCY and SON stay below one second, so growth is moderate in our test. The rows of Table 4 are not a clean test of growth, because the threshold changes between rows ($s = 3$ to $s = 12$) and a higher s means less work; for example, plain SON is faster at 24 402 baskets than at 10 000. Also, each time is a single run rounded to 0.1 s.

Spark. Spark returns the same exact result as plain SON, but in local mode on one machine it is slower: 2.2 to 7.8 s against 0.1 to 1.1 s (Table 5). Spark has a fixed cost for each job (starting tasks, moving data between Python and the JVM, shuffling counts), which is larger than the actual work on data this small; the 5 000-basket row is the slowest because $s = 3$ gives a very large candidate set to merge and broadcast. Its advantage is that the same code can run on a cluster for data that does not fit on one machine, which we could not test here.

Limitations. The Top-1000 dataset is small (1000 movies, at most four stars), so itemsets are small and supports are low. The 25k dataset has only 24 402 baskets, which is a modest test of scalability, The timings come from single runs on a Colab machine, and all baskets are kept in memory.

8 Reproducibility

Open the notebook in Google Colab and run all cells in order. The random seed is fixed (42); the samples of the scalability test depend on the order in which the cells are run. The external inputs are the two Kaggle datasets, downloaded with the Kaggle API; put your own credentials in the first cell (they are `xxxxxx` in the submitted version). Spark is installed with `pip install pyspark`. Repository: https://github.com/Abdelrahman-SS/Market-basket-analysis.

9 Declaration

"I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study."